

LAPORAN MODUL 13

PRAKTIKUM

STRUKTUR DATA



Disusun oleh :

Nama : Irvan Cahya Nugraha

NIM : 245410034

Kelas : Informatika 1

PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025

LAPORAN STRUKTUR DATA

MODUL 13

POHON BINER

A. TUJUAN PRAKTIKUM

Mahasiswa dapat mengimplementasikan penggunaan Simpul milik Double Linked List untuk pembuatan pohon

B. DASAR TEORI

Pohon (*tree*) adalah kumpulan akar (*root*) , cabang, dan simpul (*node*) yang saling terhubung secara hirarki

Pohon Biner (*binary tree*) adalah pohon dimana setiap simpulnya (*node*) hanya boleh memiliki **maksimal** 2 anak (dari cabang kiri, dan kanan)

Simpul (*node*) :

- simpul anak (*children*) hanya boleh punya 1 parent
- simpul orangtua (*parent*) hanya boleh punya **maksimal** 2 anak, namun boleh juga tidak punya anak

Simpul **Akar** adalah simpul yang tidak memiliki *parent*

Simpul **Daun** adalah simpul yang tidak memiliki anak (*children*) Cabang di dalam pohon biner terdiri dari **cabang kiri, cabang kanan**

Level menunjukkan tingkat hirarki

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1

```
PreOrder: M D A D K G F J H L P N O S W T U
```

```

class simpul {
    String elemen;
    simpul kiri, kanan;

    public simpul(String elemen) {
        this.elemen = elemen;
        this.kiri = null;
        this.kanan = null;
    }
}

class pohon {
    simpul akar;

    void deklarasiPohon () {
        akar = null;
    }

    simpul tambahSimpul (simpul Penunjuk, String ELEMEN) {
        if (Penunjuk == null) {
            Penunjuk = new simpul(ELEMEN);
        } else {
            if (ELEMEN.compareTo(Penunjuk.elemen) < 0) {
                Penunjuk.kiri = tambahSimpul (Penunjuk.kiri, ELEMEN);
            } else {
                Penunjuk.kanan = tambahSimpul (Penunjuk.kanan, ELEMEN);
            }
        }
        return Penunjuk;
    }

    void preOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            System.out.print(Penunjuk.elemen + " ");
            preOrder(Penunjuk.kiri);
            preOrder(Penunjuk.kanan);
        }
    }

    void inOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            inOrder(Penunjuk.kiri);
            System.out.print(Penunjuk.elemen + " ");
            inOrder(Penunjuk.kanan);
        }
    }

    void postOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            postOrder(Penunjuk.kiri);
            postOrder(Penunjuk.kanan);
            System.out.print(Penunjuk.elemen + " ");
        }
    }
}

public class ProgramPohonBiner {
    public static void main(String[] args) {
        pohon tree = new pohon();
        tree.deklarasiPohon ();

        tree.akar = tree.tambahSimpul (tree.akar, "M");
        tree.akar = tree.tambahSimpul (tree.akar, "D");
        tree.akar = tree.tambahSimpul (tree.akar, "P");
        tree.akar = tree.tambahSimpul (tree.akar, "D");
        tree.akar = tree.tambahSimpul (tree.akar, "A");
        tree.akar = tree.tambahSimpul (tree.akar, "S");
        tree.akar = tree.tambahSimpul (tree.akar, "K");
        tree.akar = tree.tambahSimpul (tree.akar, "N");
        tree.akar = tree.tambahSimpul (tree.akar, "G");
        tree.akar = tree.tambahSimpul (tree.akar, "O");
        tree.akar = tree.tambahSimpul (tree.akar, "L");
        tree.akar = tree.tambahSimpul (tree.akar, "W");
        tree.akar = tree.tambahSimpul (tree.akar, "F");
        tree.akar = tree.tambahSimpul (tree.akar, "J");
        tree.akar = tree.tambahSimpul (tree.akar, "T");
        tree.akar = tree.tambahSimpul (tree.akar, "H");
        tree.akar = tree.tambahSimpul (tree.akar, "U");

        System.out.print("PreOrder: ");
        tree.preOrder(tree.akar);
        System.out.println();

        System.out.print("InOrder: ");
        tree.inOrder(tree.akar);
        System.out.println();

        System.out.print("PostOrder: ");
        tree.postOrder(tree.akar);
        System.out.println();
    }
}

```

Pembahasan : Pada praktik pertama, pohon biner dibangun menggunakan metode `tambahSimpul()` yang bekerja berdasarkan prinsip *Binary Search Tree* (BST), yaitu data yang lebih kecil dari simpul penunjuk akan ditempatkan di cabang kiri, sedangkan data yang lebih besar atau sama ditempatkan di cabang kanan. Saat perintah `preOrder(akar)` dijalankan, proses penelusuran dilakukan dengan urutan akar → kiri → kanan. Hasil yang diperoleh mencerminkan struktur pohon apa adanya, karena simpul akar selalu dicetak terlebih dahulu sebelum turun ke subtree kiri dan kanan. Hal ini terjadi karena metode `preOrder()` secara eksplisit mencetak elemen simpul sebelum pemanggilan rekursif ke anak kiri dan kanan.

Praktik 2

```
InOrder: A D D F G H J K L M N
```

Pembahasan : Pada praktik kedua, perintah `inOrder(akar)` digunakan untuk mencetak isi pohon biner dengan urutan kiri → akar → kanan. Karena pohon dibentuk dengan aturan BST, hasil cetak *inOrder* akan menghasilkan data yang terurut secara menaik (ascending), baik pada versi elemen bertipe String maupun int. Hal ini terjadi karena setiap simpul kiri selalu memiliki nilai lebih kecil dari simpul induknya, dan simpul kanan selalu bernilai lebih besar, sehingga traversal *inOrder* secara alami menyusun data dari yang paling kecil hingga paling besar.

Praktik 3

```
PostOrder: A F H J G L K D D C
```

Pembahasan : Pada praktik ketiga, digunakan metode `postOrder(akar)` yang melakukan penelusuran dengan urutan kiri → kanan → akar. Hasil yang diperoleh menunjukkan bahwa simpul akar selalu dicetak paling akhir setelah seluruh subtree kiri dan kanan selesai diproses. Pola ini sesuai dengan konsep *postOrder traversal*, yang sering digunakan pada kasus penghapusan pohon atau evaluasi ekspresi, karena memastikan semua simpul anak diproses terlebih dahulu sebelum simpul induknya.

LATIHAN

Latihan 1

TUGAS

Tugas 1

```

class simpul {
    String elemen;
    simpul kiri, kanan;

    public simpul(String elemen) {
        this.elemen = elemen;
        this.kiri = null;
        this.kanan = null;
    }
}

class pohon {
    simpul akar;

    void deklarasiPohon () {
        akar = null;
    }

    simpul tambahSimpul (simpul Penunjuk, String ELEMEN) {
        if (Penunjuk == null) {
            Penunjuk = new simpul(ELEMEN);
        } else {
            if (ELEMEN.compareTo(Penunjuk.elemen) < 0) {
                Penunjuk.kiri = tambahSimpul (Penunjuk.kiri, ELEMEN);
            } else {
                Penunjuk.kanan = tambahSimpul (Penunjuk.kanan, ELEMEN);
            }
        }
        return Penunjuk;
    }

    void preOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            System.out.print(Penunjuk.elemen + " ");
            preOrder(Penunjuk.kiri);
            preOrder(Penunjuk.kanan);
        }
    }

    void inOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            inOrder(Penunjuk.kiri);
            System.out.print(Penunjuk.elemen + " ");
            inOrder(Penunjuk.kanan);
        }
    }

    void postOrder(simpul Penunjuk) {
        if (Penunjuk != null) {
            postOrder(Penunjuk.kiri);
            postOrder(Penunjuk.kanan);
            System.out.print(Penunjuk.elemen + " ");
        }
    }
}

public class ProgramPohonBine {
    public static void main(String[] args) {
        pohon tree = new pohon();
        tree.deklarasiPohon ();

        tree.akar = tree.tambahSimpul (tree.akar, "M");
        tree.akar = tree.tambahSimpul (tree.akar, "D");
        tree.akar = tree.tambahSimpul (tree.akar, "P");
        tree.akar = tree.tambahSimpul (tree.akar, "D");
        tree.akar = tree.tambahSimpul (tree.akar, "A");
        tree.akar = tree.tambahSimpul (tree.akar, "S");
        tree.akar = tree.tambahSimpul (tree.akar, "K");
        tree.akar = tree.tambahSimpul (tree.akar, "N");
        tree.akar = tree.tambahSimpul (tree.akar, "G");
        tree.akar = tree.tambahSimpul (tree.akar, "O");
        tree.akar = tree.tambahSimpul (tree.akar, "L");
        tree.akar = tree.tambahSimpul (tree.akar, "W");
        tree.akar = tree.tambahSimpul (tree.akar, "F");
        tree.akar = tree.tambahSimpul (tree.akar, "J");
        tree.akar = tree.tambahSimpul (tree.akar, "T");
        tree.akar = tree.tambahSimpul (tree.akar, "H");
        tree.akar = tree.tambahSimpul (tree.akar, "U");

        System.out.print("PreOrder: ");
        tree.preOrder(tree.akar);
        System.out.println();

        System.out.print("InOrder: ");
        tree.inOrder(tree.akar);
        System.out.println();

        System.out.print("PostOrder: ");
        tree.postOrder(tree.akar);
        System.out.println();
    }
}

```

```
C:\WINDOWS\system32\cmd. x + v
PreOrder: 50 30 20 10 40 70 60 80 100 90
InOrder: 10 20 30 40 50 60 70 80 90 100
PostOrder: 10 20 40 30 60 90 100 80 70 50
Press any key to continue . . . |
```

Pembahasan : Pada tugas ini, modifikasi menjadi angka di mana setiap simpul menyimpan satu data bertipe int pada variabel elemen serta memiliki dua referensi, yaitu kiri dan kanan. Proses penambahan data dilakukan melalui method `tambahSimpul()`, yang secara rekursif membandingkan nilai ELEMEN dengan `Penunjuk.elemen`, sehingga nilai yang lebih kecil ditempatkan pada cabang kiri dan nilai yang lebih besar atau sama ditempatkan pada cabang kanan. Dengan urutan data yang dimasukkan (50, 30, 70, 20, 40, 60, 80, 100, 10, 90), terbentuk struktur pohon yang mengikuti aturan BST.

Pada saat method `preOrder(akar)` dijalankan, proses penelusuran dilakukan dengan urutan akar, kemudian subtree kiri, lalu subtree kanan, sehingga hasil cetak mencerminkan urutan pembentukan pohon dari simpul akar ke bawah. Selanjutnya, method `inOrder(akar)` menelusuri subtree kiri terlebih dahulu, kemudian mencetak simpul induk, dan diakhiri subtree kanan, sehingga hasil yang diperoleh berupa data yang tersusun menaik karena sifat BST yang menempatkan nilai kecil di kiri dan nilai besar di kanan. Terakhir, method `postOrder(akar)` melakukan penelusuran subtree kiri dan kanan terlebih dahulu sebelum mencetak simpul induk, sehingga simpul akar selalu dicetak paling akhir.

D. KESIMPULAN

perbedaan hasil keluaran pada *preOrder*, *inOrder*, dan *postOrder* sepenuhnya ditentukan oleh urutan penelusuran simpul pada pohon biner. *PreOrder* menampilkan struktur pohon dari akar ke daun, *inOrder* menghasilkan data terurut karena sifat *Binary Search Tree*, sedangkan *postOrder* menampilkan simpul akar paling akhir. Ketiga traversal ini memberikan sudut pandang yang berbeda terhadap struktur pohon biner meskipun data dan pohonnya sama.

E. LAMPIRAN

